Kernel Hacking 4 – OOPS e sysfs



Indice

- Kernel OOPS
  - Offensive programming con BUG_ON
  - Debugging degli OOPS
- Utilizzo del *sysfs*

Approfondimento kernel log

- Studiare la sezione APPROFONDIMENTO KERNEL LOG in *3-kernel-log-oops*
- Esercizio
  - Redirezionare il kernel log sulla porta seriale emulata della macchina virtuale
  - Leggere il kernel log dalla porta seriale della macchina virtuale

Kernel OOPS e debugging

- Studiare dalla sezione KERNEL OOPS (inclusa) alla sezione DEBUGGING OOPS (inclusa), in *3-kernel-log-oops*
- Esercizio
  - Scatenare un *panic*
    - Prima al caricamento di un modulo
    - Poi provare con un panic all’avvio del kernel
      - In questo secondo caso, vi ritroverete poi con un immagine kernel inutilizzabile
  - Per entrambi i casi, ritrovare il punto del codice che ha causato il *panic*

Soluzione

- Causare il panic causando con una eccezione in `start_kernel`, per esempio
  - Divisione per zero
  - Accesso ad indirizzo illegale (per esempio in red zone)
- Prendere nota dell'indirizzo nell'*instruction pointer* riportato nell'OOPS
  - Se non si vede la riga contenente il valore di RIP sullo schermo, sfruttare il redirezionamento su porta seriale per poter leggere l'OOPS per intero
- Scoprire se la funzione riportata nell'indirizzo è definita in un modulo dinamico
- Se la funzione non è definita in un modulo dinamico
  - Aprire con `gdb` l'immagine del kernel
- Se la funzione è definita in un modulo dinamico
  - Aprire con `gdb` il file oggetto del modulo
- Usare il comando `list` per trovare la riga di codice che ha causato il panic nel codice

Indice

- Kernel OOPS
  - Offensive programming con BUG_ON
  - Debugging degli OOPS
- Utilizzo del *sysfs*

Kobject e sysfs

- La documentazione purtroppo non fornisce una guida semplice e chiara
- Mettiamo assieme i pezzi e poi facciamo il punto assieme
- Studiamo
  - https://docs.kernel.org/core-api/kobject.html
    - Sezione introduttiva fino a *Embedding kobjects* esclusa
  - https://docs.kernel.org/filesystems/sysfs.html
    - Fino a *Subsystem Specific Callbacks* esclusa
  - Poi torniamo su https://docs.kernel.org/core-api/kobject.html
    - Sezione Creating “simple” kobjects

Come funziona: creazione kobject

- Se si crea un *kobject*, si fa nascere una nuova directory in `/sys`
  - La funzione di creazione si aspetta due parametri: una stringa ed un puntatore ad un *kobject*
  - La stringa contiene il nome che sarà dato alla directory che verrà creata in `/sys`
  - Il puntatore deve puntare al *kobject* padre per il *kobject* che si sta creando
    - Questo stabilirà anche dove appare la directory che sarà creata: sarà figlia della directory associata al *kobject* padre

Come funziona: creazione file

- Ad un *kobject* si possono associare attributi
  - Ogni attributo è una stringa che si può leggere o scrivere da userspace
    - I diritti di lettura/scrittura sono configurabili
  - L’oggetto userspace attraverso il quale si può leggere/scrivere un attributo, ossia si può leggere/scrivere la stringa che rappresenta il valore dell’attributo, è un file di testo
  - Per ciascun attributo, per utilizzare il file di testo associato all’attributo, tale file va creato esplicitamente
    - C’è anche una funzione per creare direttamente un gruppo di file, ma ci basta padroneggiare la funzione per creare un singolo file
  - Dato il file creato per un dato attributo, e dato il *kobject* che contiene tale attributo, il file apparirà come figlio della directory associata a tale *kobject*

Rimozione kobject

- Riprendiamo https://docs.kernel.org/core-api/kobject.html
  - Primo paragrafo della sezione *Kobject removal*

Esempio

- Studiare e capire fino in fondo l'esempio in `samples/kobject/kobject-example.c`
- In particolare, trovare e capire le definizioni delle funzioni invocate nel corpo delle funzioni di *show* e *store*

`Hello World` con sysfs

- Estendere l'ultimo modulo che avete realizzato, affinché
  - Crei un file *hello* nel `sysfs`. Tale file:
    - Se viene letto, appare come contenere la stringa `Hello World`
    - Se viene scritto, copia nel kernel log ciò che gli viene scritto dentro

Estensione ulteriore

- Aggiungere al modulo anche una variabile intera, che può essere letta o scritta attraverso `sysfs`

Soluzione

- Una possibile soluzione è in questo file sorgente:

  https://github.com/akoskovacs/kmod_example/blob/master/hello.c

Esercizio senza soluzione

- Creare un file in `/sys` attraverso il quale si può leggere il periodo del *tick* del kernel. Non e' richiesto anche di poter modificare tale periodo
  - Un aiuto nella prossima slide

Aiuto

- Come abbiamo visto, la frequenza del kernel è controllata attraverso una opzione del config, che si chiama `CONFIG_HZ`
- Da lì provate a dedurre come si potrebbe chiamare una qualche variabile o macro globale che contiene appunto tale frequenza

Esercizio senza soluzione

- Implementare un modulo che definisce una variabile intera globale ed un file speciale che, alla sua lettura, compie le seguenti azioni:
  - Stampa il valore della variabile
  - Lo incrementa
  - Lo ristampa
  - Stampa una riga di trattini:

- Collaudare il modulo lanciando vari processi in parallelo, che fanno tutti la stessa cosa: stampare all'infinito (o almeno per tantissime iterazioni) il contenuto del file speciale su *stdout*. Lanciare ciascun processo in un terminale virtuale separato, oppure redirottare l'output di ciascun processo in un file separato
  - Controllare che le stampe dei valori della variabile siano tutte coerenti, ossia ogni coppia di stampe di valori della variabile, stampi correttamente due valori consecutivi (leggere commenti nella prossima slide solo dopo aver provato)

Stampa incoerente

- Probabilmente vi saranno delle stampe incoerenti, in cui la seconda stampa non riporta il valore successivo della variabile
- Sapete dire perché?

Corsa critica (race condition)

- L’operazione di incremento della variabile non è atomica
  - Ossia non è garantito che, durante la sua esecuzione, nessuno stato intermedio sia visibile o modificabile da alcun altra operazione
- Al contrario l’incremento è tipicamente costituito dalle seguenti tre operazioni atomiche
  - una lettura dalla memoria (del valore attuale) verso un registro della CPU
  - un incremento del valore memorizzato nel registro
  - una scrittura in memoria del nuovo valore del registro
- Le singole operazioni atomiche eseguite da due processi che stanno eseguendo l’incremento in parallelo possono mescolarsi in modi arbitrari

Esercizio

- Svolgere l’esercizio nella sezione *ESERCIZIO DA SVOLGERE DOPO AVER STUDIATO IL SYSFS* nel documento *kernel-log-oops*